

APPLIED MACHINE LEARNING TECHNIQUES · MODULE 4

Unsupervised Learning, Made Simple

Clustering · Association Rules · Evaluation Metrics — explained with everyday analogies, worked math, and zero jargon.



K-Means · GMM



Market Baskets



Cluster Quality

What We'll Cover Today

- ▶ What unsupervised learning is (and isn't)
- ▶ How it differs from supervised learning
- ▶ Real-world uses: marketing, healthcare, security
- ▶ Anomaly detection: point, contextual, collective
- ▶ Customer segmentation in business
- ▶ K-Means clustering + the Elbow Method
- ▶ Hierarchical clustering & dendrograms

- ▶ Gaussian Mixture Models & soft clustering
- ▶ Association Rule Learning
- ▶ Market basket analysis & rule mining
- ▶ Support, Confidence, and Lift — by hand
- ▶ Apriori, FP-Growth, and Eclat algorithms
- ▶ Cosine similarity, step by step
- ▶ Judging clusters: Silhouette & Davies-Bouldin

GOALS

By the End, You Will Be Able To...



Tell apart supervised and unsupervised learning — instantly, with examples.



Spot where unsupervised learning quietly runs the real world around you.



Cluster data using K-Means, Hierarchical methods, and Gaussian Mixture Models.



Mine “bought-together” rules with Apriori, FP-Growth, and Eclat.



Score cluster quality using Silhouette Score and the Davies-Bouldin Index.

Introduction to Unsupervised Learning

- The core idea — learning without an answer key
- Everyday analogies: jigsaw puzzles, fruit baskets, and bikes
- The process flow, and the three big approaches
- Supervised vs. unsupervised — side by side

What Is Unsupervised Learning?

Unsupervised learning is machine learning on data that carries **no labels and no right answers**. The model isn't told what to find — it discovers hidden patterns, groups, and relationships on its own.



Analogy: a jigsaw with no box photo

You've lost the box, so nobody can tell you what the final picture looks like. You still make progress: sky-blue pieces in one pile, straight edges in another, green textures in a third. You grouped by similarity — exactly what unsupervised learning does.



Contrast: studying with an answer key

Supervised learning is like practicing with solved papers — every question comes with the correct answer, and you learn the mapping. Unsupervised learning is handed only the questions and asked: “what structure do you see?”

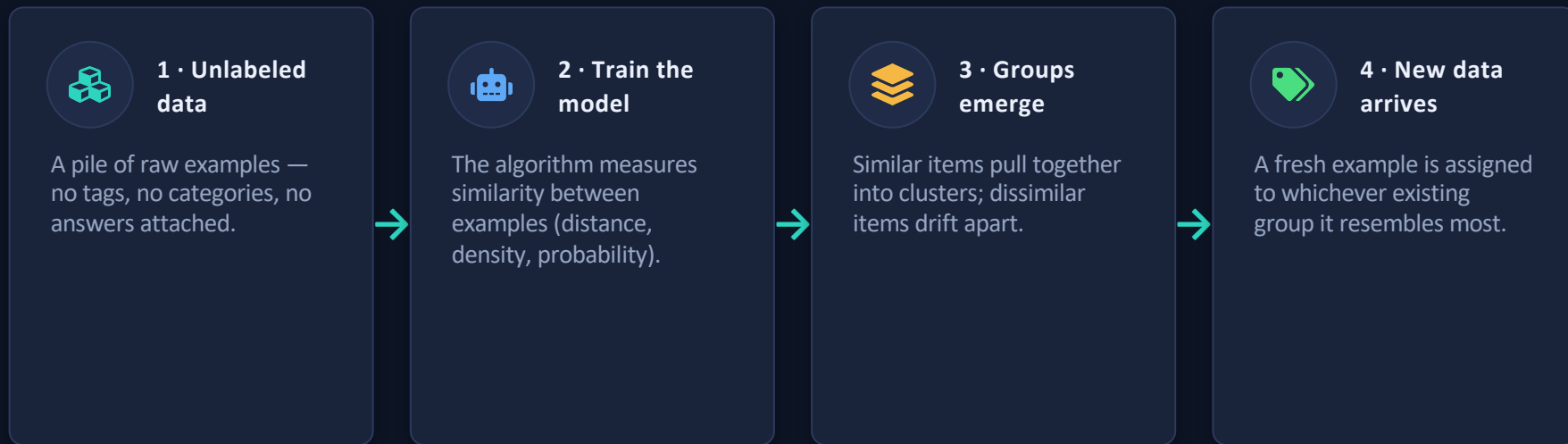
Can You Guess the Fruit From Features Alone?

No labels are given — only color, shape, and size. Your brain clusters them anyway. That instinct is the algorithm.

Color	Shape	Size	Your brain says...
Yellow	Long, curved cylinder	Big	Banana
Red	Round, dip on top	Big	Apple
Green / purple	Small ovals in a bunch	Small	Grapes

Key point: you never saw a label saying “banana”. You formed groups purely from shared features — and a name for each group came *after* the grouping. Unsupervised models work in exactly this order: cluster first, interpret later.

The Unsupervised Learning Process Flow



Note: “training data” here just means a collection of information **without any labels** — the model is never shown a correct answer at any point.

WORKED EXAMPLE

Sorting Vehicles Without Being Told What They Are

- 1 Feed in a mixed set of two-wheelers — bicycles, scooters, sports bikes — described only by features: size, top speed, engine, color.
- 2 The system notices natural groupings: small + pedal-powered, medium + step-through, large + high speed — and forms categories on its own.
- 3 Show it a brand-new vehicle: it compares features and drops it into the most similar existing group.

The groups depend entirely on the attributes you choose. Feed in color → you get red vs. green piles. Feed in speed → racing vs. commuter piles. Same data, different lens, different clusters. Feature choice is the steering wheel of unsupervised learning.

Three Families of Unsupervised Learning



Clustering

Group similar things together.

Like sorting a laundry basket: whites, colors, delicates — no one labeled the clothes, the similarity did the work.

Examples: K-Means, GMM, Hierarchical.



Association Rules

Find what goes together.

Like noticing chai is almost always ordered with biscuits — a “if A, then probably B” pattern.

Examples: Apriori, FP-Growth, Eclat.



Dimensionality Reduction

Compress many features into few.

Like a movie trailer: 2 minutes that keep the story of 2 hours. Less data, same essence.

Examples: PCA, t-SNE, Autoencoders.

This module focuses on the first two — clustering and association rules.

Supervised vs. Unsupervised Learning

Aspect	Supervised	Unsupervised
Data	Labeled — every example has an answer	Unlabeled — raw examples only
Goal	Predict outcomes (classify / regress)	Discover structure (clusters / rules)
Human effort	Heavy — someone must label the data	Light — no labeling needed
Algorithms	Linear Regression, SVM, Decision Trees	K-Means, PCA, Hierarchical, Apriori
Typical use	Spam detection, fraud prediction	Customer segments, topic discovery

One-line memory hook: supervised = exam with an answer key; unsupervised = detective work with no key, only clues.

Same Data, Different Question

Supervised

You get pairs: $(x_1, y_1), (x_2, y_2), \dots$

Learn a function $f: X \rightarrow y$ that maps inputs to known answers.

Unsupervised

You get only inputs: $x_1, x_2, \dots, x_n$

Learn the **structure of X itself** — groups, co-occurrence rules, or a compressed form.

Concrete example — same spreadsheet of customers:

Supervised question: “Given past churners (labels), will THIS customer churn?”

Unsupervised question: “With no labels at all, what natural customer groups exist in here?”

PART 2

02

Where It's Used in the Real World

- Five everyday applications you already interact with
- Deep dives: healthcare, marketing, cybersecurity
- The honest list of challenges

It's Already All Around You



News grouping

Google News bundles hundreds of articles about the same event — nobody labels them; similarity of words does it.



Computer vision

Grouping visually similar images or regions — e.g., photo apps clustering faces without knowing names.



Medical imaging

Separating tissue types in MRI/CT scans by pixel patterns, helping flag unusual regions.



Anomaly detection

Learning what “normal” looks like, then raising a flag when something deviates — fraud, intrusions, faults.



Recommendations

“Customers like you also bought...” — grouping users by behavior, not by declared identity.

USE CASE 1

Healthcare — Finding Patient Groups No One Defined

Hospitals cluster patients by patterns in vitals, lab results, and history — **without predefined risk labels** — and discover risk segments the staff never explicitly created.

Cluster found	Pattern (BP, cholesterol)	What doctors do with it
Group A	Low BP, low cholesterol	Routine annual checkups
Group B	Borderline values, drifting upward	Lifestyle program + 6-month review
Group C	High BP and high cholesterol together	Priority screening & medication review

The win: the “high-risk” label was never in the data — the cluster surfaced it, and humans named it afterwards.

Marketing — Customer Segmentation

Group customers by behavior — purchase frequency, average spend, browsing — **with no predefined segments**. The algorithm draws the boundaries; marketing names them.

Customer	Visits / month	Avg spend (₹)	Cluster → nickname
C1	13	3,100	Frequent big spenders
C2	2	420	Occasional shoppers
C3	7	1,100	Price-sensitive regulars
C4	14	3,250	Frequent big spenders

Payoff: send VIP perks to big spenders, discount coupons to the price-sensitive — same budget, far better conversion.

Cybersecurity — Catching Attacks Nobody Has Seen

The model learns what **normal** user and network behavior looks like. Anything that lands far from that normal cloud gets flagged — **no labeled attack examples required.**



Normal looks like...

Sessions of 80–120 seconds moving 400–600 KB, clustered tightly together during office hours. Dense, boring, predictable — exactly what we want.



Suspicious looks like...

A 3 a.m. session moving 950 KB in 30 seconds, sitting far outside the dense cloud. The model has never seen this attack before — distance alone betrays it.

Why this matters: supervised systems only catch attacks they were trained on; unsupervised systems can catch attacks invented yesterday.

Five Challenges of Unsupervised Learning



Computational cost: Comparing everything with everything is expensive — big data makes it heavier.



Longer training time: No answer key means more exploration before the structure settles.



Risk of wrong results: With no ground truth, the model can group things confidently — and wrongly.



Humans must validate: Someone has to inspect clusters and decide if they actually mean anything.



Low transparency: It's often unclear why two items landed in the same cluster — explainability suffers.

Business Applications

- Anomaly detection — the art of spotting the odd one out
- Three flavors: point, contextual, and collective anomalies
- Customer segmentation as a business engine

What Is Anomaly Detection?

Anomaly detection is the process of identifying data points, patterns, or events that deviate significantly from what's normal. Think of it as a school assembly in uniform — **one student in pyjamas is instantly visible**.



A sudden stock-price drop

A share that trades calmly at ₹500 $\pm$ 10 all month plunges to ₹320 in an hour. Nothing in recent history resembles it — anomaly.



A huge payment, brand-new user

An account created yesterday sends ₹5,00,000 abroad as its very first action. Each detail might be fine alone — together, suspicious.

Coming up: anomalies come in three flavors — point, contextual, and collective. Knowing which one you face changes how you detect it.

Point Anomaly — One Value Screams

A **single data point** that deviates sharply from everything else — no context needed, the value alone is wild.

The math: a quick z-score

Purchases usually range ₹20–₹100: mean $\mu = 60$, spread $\sigma = 20$.

A ₹10,000 purchase appears. How odd is it?

$$z = (x - \mu) / \sigma = (10,000 - 60) / 20 \approx 497$$

Typical points sit within $z = \pm 3$. This one is ~497 standard deviations out — not unusual, **impossible-by-accident**.



Fraud detection



Card transactions



Sensor spikes (CPU at 100% when normal is 10–20%)

Contextual Anomaly — Normal Here, Alarming There

The same value can be perfectly normal in one context and a red flag in another. The context — time, season, situation — decides.

35°C in May (summer)

Completely expected in Pune. Nobody blinks. Same number, friendly context.

35°C in December (winter)

Highly anomalous — the value is identical, but the seasonal context makes it an outlier.

More examples: heart rate 150 bpm while running = healthy; 150 bpm while asleep = emergency. A ₹50,000 card swipe during Diwali shopping = plausible; the same swipe at 4 a.m. on a Tuesday = review it.

Where it shows up: time-series monitoring, weather data, health trackers.

Collective Anomaly — Innocent Alone, Guilty Together

A group of related points is anomalous as a pattern, even though each individual point looks fine on its own.

The classic example:

1 failed login → someone fat-fingered a password. Normal.

50 failed logins in 2 minutes on one account → **brute-force attack**.

Every single attempt is an ordinary event — the rapid sequence is the anomaly.



Cybersecurity



IoT & sensor streams



Network bursts at 3 a.m. (non-peak hours)

Quick recall: Point = the value is weird. Contextual = the value is weird here/now. Collective = the values are weird together.

Customer Segmentation

Customer segmentation divides customers into distinct groups by shared behavior — purchase patterns, frequency, demographics — typically using K-Means clustering on transaction data.

Spend →



Visits per month →



Frequent high-spenders

Reward & retain — VIP tiers, early access.



Infrequent budget buyers

Nudge with coupons and reminders.



Window shoppers

Convert with first-purchase offers.

Clustering Techniques

- What clustering really optimizes
- K-Means step by step — including the math by hand
- Choosing K with the Elbow Method
- Hierarchical clustering, dendrograms, GMM & soft clustering

What Is Clustering?

Clustering groups unlabeled examples so that items in the same group are highly similar, and items in different groups are clearly different. Similarity is usually measured as **distance** — closer points belong together.

Analogy: the messy wardrobe

Dump every garment on the bed. Without any rulebook, you naturally make piles: shirts with shirts, jeans with jeans, winterwear together.

Two instincts drive you — and drive every clustering algorithm:

1. items inside a pile should match; 2. piles should differ from each other.

The formal goal

Minimize **intra-cluster distance** (tight groups)

Maximize **inter-cluster distance** (separated groups)

Distance between two points (Euclidean):

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$\text{e.g., } d((2,3),(5,7)) = \sqrt{(9+16)} = \sqrt{25} = 5$$

K-Means — The Pizza Outlet Strategy

K-Means splits data into K groups by repeatedly moving K “center points” (centroids) until every point sits with its nearest center. Analogy: place K pizza outlets in a city, then keep relocating each outlet to the middle of its own customers.

- 1 Choose K — how many clusters (outlets) you want.
- 2 Drop K centroids at random starting spots.
- 3 Assign every point to its nearest centroid.
- 4 Move each centroid to the average position of its assigned points.
- 5 Repeat 3–4 until centroids stop moving — the map has stabilized.

K-Means on Six Numbers — Watch It Converge

Data (customer visits/month): **2, 3, 4, 10, 11, 12** We want $K = 2$ clusters.

Step 1 · Pick starting centroids Choose any two points, say $c_1 = 2$ and $c_2 = 12$.

Step 2 · Assign by nearest centroid Point 3 $\rightarrow c_1$ ($|3-2|=1 < |3-12|=9$). Point 10 $\rightarrow c_2$ ($|10-12|=2 < 8$). Result: {2, 3, 4} and {10, 11, 12}.

Step 3 · Move centroids to the mean $c_1 = (2+3+4) \div 3 = 3$ $c_2 = (10+11+12) \div 3 = 11$

Step 4 · Re-check assignments Every point still picks the same centroid $\rightarrow$ nothing changes $\rightarrow$ converged in one pass.

Final answer: Cluster A = {2, 3, 4} around centroid 3; Cluster B = {10, 11, 12} around centroid 11. Real datasets need more rounds — the loop is identical.

The Elbow Method

Problem: K-Means needs K up front — but how many clusters does the data *really* have? **Answer:** plot WCSS (within-cluster sum of squares — total squared distance of points to their centroid) for K = 1...10 and look for the bend.

How to read it

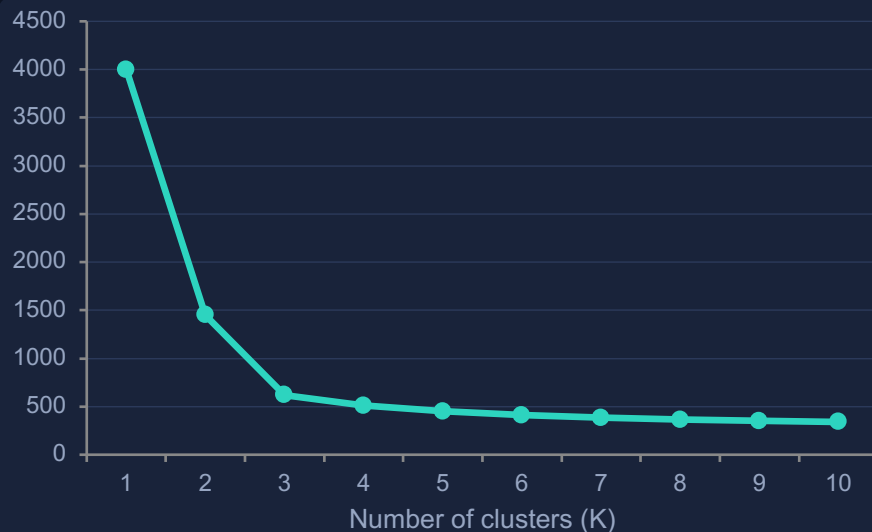
K = 1 → 2: WCSS crashes (huge gain)

K = 2 → 3: still a big drop

K = 3 → 4+: curve flattens — extra clusters barely help

The bend at K = 3 is the “elbow” → pick 3.

Analogy: hiring delivery staff — the 2nd and 3rd hire transform speed; the 7th barely moves the needle.



Hierarchical Clustering — Building a Family Tree of Data

Hierarchical clustering builds a tree of clusters by progressively merging or splitting groups. Big advantage: **you don't have to choose K in advance** — you cut the finished tree wherever it makes sense.



Agglomerative (bottom-up)

Every point starts as its own tiny cluster. At each step, the two closest clusters merge — like relatives discovering common ancestors and joining family branches, until one big family remains.

This is the common one in practice.



Divisive (top-down)

Start with everything in one giant cluster, then split recursively — like a company beginning as one team, then dividing into departments, then sub-teams.

Same tree, built from the opposite end.

Dendrograms

A **dendrogram** is the tree diagram of those merge steps. **Branch height** = how dissimilar two clusters were when they merged. Low merge = close friends; high merge = distant acquaintances.

similar $\uparrow$ height = dissimilarity



How to use it

A & B merge low $\rightarrow$ nearly identical.

The red top bar merges $\{A,B,C\}$ with $\{D,E\}$ very high $\rightarrow$ two genuinely different families.

Cut the tree with a horizontal line: cutting below the red bar gives 2 clusters; cutting lower gives 3. The tree lets you choose K after seeing the structure.

Gaussian Mixture Models (GMM)

A **GMM** assumes the data was generated by a mix of several bell curves (Gaussian / normal distributions) — one per cluster. Instead of drawing hard borders, it asks: *“which bell curve most likely produced this point?”*

Analogy: guessing by accent

Hear a stranger speak and guess their home city. You rarely say “definitely Mumbai” — you say “70% Mumbai, 30% Pune.”

GMM thinks the same way: every point gets a probability of belonging to each cluster, not a single verdict.

Why prefer it over K-Means?

- Clusters can be stretched ellipses, not just circles
- Handles overlapping groups gracefully
- Outputs confidence, not just a label

E-commerce example: segment customers by visits/month and average spend — borderline customers get split memberships instead of being forced into one box.

Soft Clustering — Allowing Mixed Membership

Soft (fuzzy) clustering lets one data point belong to several clusters at once, each with a degree of membership (a probability), instead of exactly one.

Hard clustering (K-Means)

Customer A → Budget Shopper: **100%**

One box per point. Clean, but blunt for borderline cases.

Soft clustering (GMM)

Customer A → Budget **70%** · Moderate **30%**

Honest about uncertainty — useful when groups genuinely blur.

Analogy: a gym-goer who lifts weights 70% of visits and swims 30% — calling them “only a lifter” loses real information. Soft clustering keeps it.



HANDS-ON DEMONSTRATION

Customer Segmentation using K-Means, Hierarchical & GMM

Follow along: open Module 4 → Demo 1 on the LMS for the dataset, notebook, and detailed steps.

Association Rule Learning

- Market basket analysis — the “bought-together” science
- Support, Confidence & Lift — every formula worked by hand
- Apriori, FP-Growth, and Eclat — three ways to mine rules

Association Rule Mining

Association rule mining finds interesting “goes-together” relationships among items in transaction data. Rules take the form $\{A\} \rightarrow \{B\}$: “if a basket contains A, it likely contains B too.”



Market basket analysis

What sells together in stores



Recommender systems

“Frequently bought together”



Web usage mining

Pages visited in the same session



Bioinformatics

Genes that activate together

Unlike clustering (which groups customers), association rules connect products.

Market Basket Analysis

A supermarket studies which products land in the same basket so it can **optimize shelf placement, design bundles, and power checkout suggestions.**

“If a customer buys bread, they are likely to buy butter.”

Action: place butter one shelf away from bread — not across the store.

“If a customer buys pasta and sauce, they are likely to buy cheese.”

Action: bundle the trio — “Italian night combo” — at checkout.

Famous (and fun) anecdote: a US retailer reportedly found diapers and beer selling together on Friday evenings — young fathers on a last errand. True or legend, it captures the spirit: the data reveals pairings no manager would guess.

Three Questions, Three Metrics

We'll judge the rule $\{\text{Milk}\} \rightarrow \{\text{Bread}\}$ on this tiny shop ledger of 5 baskets — and reuse it for every calculation.

Basket	Milk	Bread	Butter
T1	✓	✓	✓
T2	✓	✓	✗
T3	✗	✓	✓
T4	✓	✗	✓
T5	✓	✓	✓

Support

“How common is the combo overall?” — popularity of A and B appearing together.

Confidence

“When A is bought, how often does B follow?” — reliability of the rule.

Lift

“Is the connection real, or just two popular items coinciding?” — strength vs. pure chance.

Read the table once now — T1, T3, T5 have milk AND bread; T2 also does; T4 has milk but no bread.

METRIC 1

Support — “How Common Is the Combo?”

$\text{Support}(A \rightarrow B) = \text{baskets containing both A and B} \div \text{total baskets}$

Basket	Milk	Bread	Butter
T1	✓	✓	✓
T2	✓	✓	✗
T3	✗	✓	✓
T4	✓	✗	✓
T5	✓	✓	✓

Worked example: {Milk} → {Bread}

Baskets with BOTH milk & bread: T1, T2, T5 → 3

Total baskets: 5

Support = 3 / 5 = 0.6

Plain words: 60% of all shopping trips include this pair — it's a common combo, worth attention.

Use it to: filter out rare combos early — a rule that appears in 0.1% of baskets isn't worth shelf space.

Confidence — “When A Happens, Does B Follow?”

$\text{Confidence}(A \rightarrow B) = \text{baskets containing both A and B} \div \text{baskets containing A}$

Basket	Milk	Bread	Butter
T1	✓	✓	✓
T2	✓	✓	✗
T3	✗	✓	✓
T4	✓	✗	✓
T5	✓	✓	✓

Worked example: {Milk} → {Bread}

Baskets with milk: T1, T2, T4, T5 → 4

Of those, also containing bread: T1, T2, T5 → 3

Confidence = 3 / 4 = 0.75

Plain words: among milk buyers, 75% also pick up bread. The rule is fairly reliable.

Careful: confidence alone can mislead — if bread is in nearly every basket anyway, ANY rule pointing to bread looks confident.
Enter Lift...

Lift — “Is the Link Real, or Just Coincidence?”

$\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) \div \text{Support}(B)$ — how much A boosts B beyond B's normal popularity

Worked example: {Milk} → {Bread}

Confidence = 0.75 (previous slide)

Support(Bread) = $4/5 = 0.8$ (T1,T2,T3,T5)

$\text{Lift} = 0.75 / 0.8 = 0.9375$

Milk buyers buy bread slightly LESS than shoppers in general — the “rule” adds nothing.

Lift > 1 A genuinely pulls B into the basket — a real, useful association. Act on it.

Lift = 1 A and B are independent — they co-occur exactly as often as chance predicts.

Lift < 1 A slightly discourages B — buyers of A buy B less than average. (Our milk→bread case: 0.94.)

Rule of thumb: a rule earns shelf space only when support is decent, confidence is high, AND lift is comfortably above 1.

The Apriori Algorithm

Apriori's one big idea: *“if an itemset is frequent, every subset of it must also be frequent.”* Flip it around for power: if {milk} alone is rare, then {milk, bread} can never be frequent — so prune it without even counting. Like a knockout tournament: eliminated teams can't reach the final.

1

Set a minimum support threshold (e.g., “must appear in $\geq 20\%$ of baskets”).

2

Count all single items; keep only the frequent ones.

3

Combine survivors into pairs, then triples — candidate k-itemsets.

4

Prune any candidate containing an infrequent subset (the trick!).

5

Repeat until no new frequent itemsets emerge.

6

Turn frequent itemsets into rules; keep those with high confidence & lift.

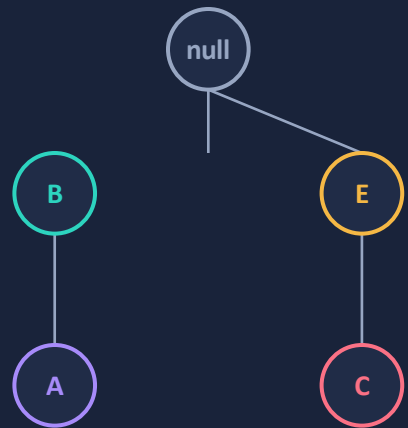
FP-Growth — Skip the Candidates Entirely

FP-Growth (Frequent Pattern Growth) finds frequent itemsets WITHOUT generating candidate sets. It compresses all transactions into one shared tree (the FP-tree) in just 2 database scans, then mines the tree recursively. Analogy: instead of re-reading a diary for every question, build an index once and query the index.

Scan 1: Count item frequencies; sort items by popularity (B:5, E:4, A:3, C:3 — drop rare D).

Scan 2: Re-insert each transaction in that order into a tree; shared prefixes share branches.

Mine: Walk the tree bottom-up with conditional sub-trees — frequent patterns fall out recursively.



shared prefixes = one branch

Apriori vs. FP-Growth

Parameter	Apriori	FP-Growth
Approach	Generate every candidate set, filter by support	Build one FP-tree, mine it recursively
Speed	Slows badly as data grows (candidates explode)	Fast — no candidate explosion
Database scans	Many — one per itemset length	Exactly 2
Memory	High when candidates pile up	Compact — shared prefixes save space
Best for	Small / sparse datasets, teaching the concept	Large, dense, real-world datasets

Memory hook: Apriori brute-forces a guest list; FP-Growth builds a family tree and reads it.

Eclat — Flip the Table Sideways

Eclat (Equivalence CLASS Transformation) stores data vertically: instead of “basket → items”, it keeps “item → list of basket IDs (TIDs)”. Counting a combo's support then becomes a simple **set intersection**.

Basket	Items
T1	milk, bread
T2	bread, butter
T3	milk, butter
T4	bread, butter



Item	TID list
milk	{T1, T3}
bread	{T1, T2, T4}
butter	{T2, T3, T4}

Support by intersection

$\text{bread} \cap \text{butter}$
 $= \{T1, T2, T4\} \cap \{T2, T3, T4\}$
 $= \{T2, T4\}$

support = 2

No re-scanning the database — every deeper combo is just another list intersection. Depth-first, fast on dense data.

Eclat vs. Apriori

Feature	Apriori	Eclat
Data layout	Horizontal — basket → items	Vertical — item → TID list
Search strategy	Breadth-first (all pairs, then all triples...)	Depth-first (drill one branch fully)
Counting support	Join + prune + re-scan database	Intersect TID lists — no re-scan
Dense data	Slower	Generally faster
Memory	Lower early on	Can grow — TID sets live in memory

Choosing among the three: Apriori to learn & for small data · FP-Growth for big retail logs · Eclat when data is dense and memory allows.

Evaluation & Similarity

- Cosine similarity — comparing direction, not size
- Silhouette Score — is each point in the right group?
- Davies-Bouldin Index — are the groups compact and far apart?

Cosine Similarity — Direction Over Magnitude

Cosine similarity measures the angle between two vectors: $\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) \div (||\mathbf{A}|| \times ||\mathbf{B}||)$. Result near 1 = same direction (similar taste); near 0 = unrelated. Size doesn't matter — only proportions do.

Intuition first

Customer A buys [5 apples, 0 oranges]
Customer B buys [10 apples, 0 oranges]

B spends double — but both buy ONLY apples. Same taste, different budget:

cosine similarity = 1 (identical direction)

Great for shopping carts & documents, where proportions matter more than totals.

Full calculation, by hand

$$\mathbf{A} = (2, 1) \quad \mathbf{B} = (1, 3)$$

$$\mathbf{A} \cdot \mathbf{B} = (2 \times 1) + (1 \times 3) = 5$$

$$||\mathbf{A}|| = \sqrt{4+1} = \sqrt{5} \approx 2.24$$

$$||\mathbf{B}|| = \sqrt{1+9} = \sqrt{10} \approx 3.16$$

$$\cos(\theta) = 5 \div (2.24 \times 3.16) \approx 0.71$$

→ fairly similar, not identical.

Silhouette Score — “Am I With the Right Crowd?”

For each point: **a** = average distance to its own cluster mates, **b** = average distance to the nearest other cluster.

$s = (b - a) \div \max(a, b)$ Range: -1 (wrong cluster) ... 0 (on the border) ... +1 (perfectly placed).

Worked example

A customer sits on average:

a = 2 units from her own segment
b = 8 units from the nearest other segment

$$s = (8 - 2) \div \max(2, 8) = 6/8 = 0.75$$

→ comfortably closer to her own group: well clustered.

Analogy: the school lunchroom

$s \approx 1$: you sit with your friends, far from other tables — clearly belong.

$s \approx 0$: you sit exactly between two tables — ambiguous.

$s < 0$: you're closer to another table than your own — you've been seated wrong.

Average s over ALL points = score of the whole clustering.

Davies-Bouldin Index — Compact and Far Apart?

The **DB index** compares each cluster's size (internal spread) to its distance from other clusters, averaged over all clusters. **Lower is better** — small, tight islands far from each other.

Low DB index ✓

Clusters are compact AND well separated — like three distant villages, each tightly built. Easy to tell apart.

High DB index ✗

Clusters are sprawling or overlapping — like suburbs bleeding into each other. Boundaries are guesswork.

Metric	Good score	Question it answers
Silhouette	High (→ +1)	Is each point with its right crowd?
Davies-Bouldin	Low (→ 0)	Are groups tight and far apart overall?



HANDS-ON DEMONSTRATION

Market Basket Analysis using Apriori & FP-Growth

Follow along: open Module 4 → Demo 2 on the LMS for the transaction dataset and step-by-step notebook.

What You Can Now Do



Distinguish supervised (answer key) from unsupervised (detective work) — and know when each fits.



Map unsupervised learning to real business wins: segmentation, anomaly detection, recommendations.



Run K-Means by hand, pick K with the elbow, read a dendrogram, and explain GMM's soft memberships.



Compute Support, Confidence & Lift on a basket table — and mine rules via Apriori, FP-Growth, Eclat.



Judge any clustering with Silhouette Score (high = good) and Davies-Bouldin Index (low = good).

Thank You

Questions, ideas, and “wait, explain that again” moments are all welcome.